

Phase-Aware Reinforcement Learning Based Page Replacement Policy for Adaptive Memory Management

CSD 204 — Operating Systems | Preliminary Report

Pranav Gupta, Shambhavi Sharma, Mohak Joshi
Shiv Nadar Institute of Eminence
pg888@snu.edu.in, ss138@snu.edu.in, mj272@snu.edu.in

March 15, 2026

1. Problem Statement

Modern workloads—databases, ML training pipelines, and web servers—exhibit temporally dynamic memory access patterns with distinct behavioral phases, such as transitions from sequential streaming to randomized lookups or from read-heavy to write-intensive epochs. Classical policies—LRU, CLOCK, and LFU—rely on static heuristics (recency, clock-hand approximation, and frequency) and cannot detect or react to such phase transitions, resulting in elevated page fault rates and suboptimal cache utilization. Belady’s optimal algorithm, while theoretically ideal, is offline and requires future knowledge unavailable at runtime. There is a critical need for an online, adaptive replacement mechanism that reconfigures its eviction strategy in response to workload phase shifts without requiring prior trace knowledge.

2. Project Overview

This project proposes a Phase-Aware Reinforcement Learning (PA-RL) page replacement algorithm with three integrated components: a phase detector, an RL agent, and a neural network value estimator. The phase detector monitors a sliding access window—tracking reuse distance distribution, access entropy, and locality score—to identify transitions between streaming, looping, and random-access phases. The RL agent is formulated as a Markov Decision Process (MDP): the state encodes per-page recency rank, frequency, reuse distance, and a phase label; the action selects a victim page; and the reward is the negative instantaneous miss rate. A lightweight PyTorch neural network serves as the Q-function approximator, trained online via Double DQN with experience replay. The system will be evaluated on a C++/Python page cache simulator using SPEC CPU2017 and YCSB traces across cache sizes of 64–4096 pages.

3. Objectives and Key Contributions

- Design an online phase-aware RL agent that adapts to workload shifts without offline pre-training.
- Develop a lightweight phase detection module using access entropy and reuse distance statistics.
- Build a high-fidelity page cache simulator with pluggable replacement policy support.
- Evaluate PA-RL against LRU, CLOCK, LFU, ARC, and Belady’s optimal on SPEC CPU2017 and YCSB.
- Demonstrate 10–25% miss rate reduction under mixed-phase workloads with statistical significance.

4. Literature Survey

4.1 Classical Page Replacement Policies

Belady [1] established the offline optimal algorithm—evict the page with the furthest next use—providing a theoretical upper bound. Denning [2] formalized the working set model, underpinning LRU’s temporal locality assumption; LRU performs well under looping workloads but degrades severely under sequential streaming (thrashing). Carr and Hennessy [3] introduced CLOCK, which approximates LRU using a circular reference-bit buffer, becoming the de facto standard in production OS kernels. LFU retains high-frequency pages but suffers from cache pollution by historically popular pages no longer in active use.

4.2 Advanced Heuristic Policies

Megiddo and Modha [4] proposed ARC, which maintains dual LRU lists for recently and frequently used pages, dynamically balancing recency and frequency without manual tuning. Jiang and Zhang [5] introduced LIRS, exploiting inter-reference recency (IRR) to retain pages with high future reuse probability, outperforming LRU on low-locality workloads. The 2Q algorithm [6] filters single-access pages via a FIFO queue before promoting to an LRU queue, reducing pollution under mixed streaming and looping patterns. Despite these improvements, all remain hand-engineered heuristics with no capacity to learn from or adapt to workload phase structure.

4.3 Machine Learning-Based Approaches

Shi et al. [7] proposed the Deep Belady Model (DBM), training a neural network offline on access traces to predict reuse distances; effective but non-adaptive by design. Zhong et al. [8] formulated cache eviction as an MDP using policy gradient learning, but assumed stationary workloads requiring offline batch training. Lykouris and Vassilvitskii [9] showed theoretically that ML predictions can augment classical policies with bounded competitive ratios. Liu et al. [10] applied LSTMs to predict access sequences (LPR), which generalizes poorly across phase transitions. Xiao et al. [11] demonstrated learning-based replacement gains in I/O-bound scenarios; Sahin et al. [12] applied RL to hybrid DRAM/NVM page allocation; and Chandio et al. [13] showed RL outperforms static heuristics in scientific computing pipelines.

4.4 Identified Research Gap

No existing policy simultaneously achieves: (1) online learning without offline pretraining, (2) explicit detection and adaptation to workload phase transitions, and (3) computational tractability for OS-level integration. PA-RL addresses this gap by coupling a lightweight phase detector with an online Double DQN agent for continuous, principled adaptation.

5. Methodology Preview

A C++/Python page cache simulator with pluggable policy support will enable deterministic trace replay. The PA-RL agent uses a three-layer PyTorch network as the Q-function approximator, trained via Double DQN (replay buffer: 10,000; target update: every 500 steps). The phase detector operates on a 256-access sliding window, computing Shannon entropy and reuse distance histograms to classify the current phase. Primary metrics—miss rate, hit rate, and average memory access time—will be reported across SPEC CPU2017 and YCSB traces.

References

- [1] L. A. Belady, “A study of replacement algorithms for a virtual-storage computer,” *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.
- [2] P. J. Denning, “The working set model for program behavior,” *Communications of the ACM*, vol. 11, no. 5, pp. 323–333, 1968.
- [3] R. W. Carr and J. L. Hennessy, “WSCLOCK—a simple and effective algorithm for virtual memory management,” *ACM SIGOPS Operating Systems Review*, vol. 15, no. 5, pp. 87–95, 1981.
- [4] N. Megiddo and D. S. Modha, “ARC: A self-tuning, low overhead replacement cache,” in *Proc. USENIX FAST*, 2003, pp. 115–130.
- [5] S. Jiang and X. Zhang, “LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance,” *ACM SIGMETRICS Performance Evaluation Review*, vol. 30, no. 1, pp. 31–42, 2002.
- [6] T. Johnson and D. Shasha, “2Q: A low overhead high performance buffer management replacement algorithm,” in *Proc. VLDB*, 1994, pp. 439–450.
- [7] Y. Shi, C. Abad, and T. Wood, “Deep Belady: Learning optimal cache replacement with neural networks,” in *Proc. IEEE HPCA*, 2020, pp. 228–241.
- [8] Y. Zhong, X. Zhang, and A. L. N. Reddy, “Reinforcement learning for cache management in heterogeneous storage systems,” *IEEE Transactions on Computers*, vol. 70, no. 8, pp. 1198–1212, 2021.
- [9] T. Lykouris and S. Vassilvitskii, “Competitive caching with machine learned advice,” *Journal of the ACM*, vol. 68, no. 4, pp. 1–25, 2021.
- [10] Z. Liu, A. Bhatotia, and R. Rodrigues, “LPR: Learned page replacement for virtual memory systems,” in *Proc. ACM EuroSys*, 2022, pp. 1–16.
- [11] Y. Xiao et al., “Learning-based page replacement scheme for efficient I/O processing,” *Scientific Reports*, vol. 15, Feb. 2025. [Online]. Available: <https://www.nature.com/articles/s41598-025-88736-4>
- [12] S. Sahin et al., “Page allocation in hybrid main memory using reinforcement learning,” *Journal of Systems Architecture*, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S1383762124002479>
- [13] A. Chandio et al., “Learning-based page replacement scheme for scientific applications,” in *Proc. ACM Conf.*, Nov. 2022. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3564695.3564777>